DA-V — Walkthrough Passo a Passo do Lado Cliente (Receiver / Consumer)

> Guia detalhado do fluxo de execução, ciclo de vida e operação do **Cliente Receptor (Client/Receiver)** no serviço **DA-V (Distribuidor Aamplo - Video)**.

---

1. Visão Geral da Arquitetura do Cliente

No DA-V, o cliente é puramente focado em **backend de recepção e reconstrução de vídeo de altíssimo desempenho e baixa latência**:

```
[Rede / UDP Socket]
        |  (Datagramas criptografados em porta UDP configurável)
        ▼
[Camada KCP ARQ]
        |  (Ordenação rápida, ACKs imediatos, retransmissão seletiva NACK)
        ▼
[Canal Criptográfico (ChaCha20-Poly1305)]
        |  (Verificação da Tag MAC de 16 bytes e decifragem do payload)
        ▼
[Motor de QoS / FEC Recovery]
        |  (Reconstrução de pacotes perdidos via paridade XOR/Reed-Solomon)
        ▼
[Adaptive Jitter Buffer (Variância de Welford)]
        |  (Absorção de oscilações de rede com delay mínimo: 5ms a 20ms)
        ▼
[Frame Assembler (Reconstrução do Wire Protocol 32B)]
        |  (Montagem de fragmentos 0..N-1 em fluxo de vídeo contínuo)
        ▼
[Bitstream H.264 Annex-B Pronto]
        |  (SPS, PPS, IDR Keyframes e P-Slices)
        ▼
[Decodificador de Vídeo / Consumer API / Display]
```

---

2. Passo a Passo de Execução Prática no Cliente

Passo 1: Obter o IP Local do Cliente

Antes de iniciar a recepção, verifique o IP da máquina do cliente na rede local:
- No Windows: `ipconfig` (procure pelo Adaptador Wi-Fi ou Ethernet, ex: `192.168.10.102` ou

`192.168.1.50`)
- No Linux: `ip a` ou `hostname -I`

Passo 2: Iniciar o Receptor (Modo Standalone)

Abra o terminal no computador cliente e execute:

```
py -3 scripts/lan_test.py --mode receiver --port 9000
```

O que o cliente faz na inicialização:

1. Cria e vincula o socket UDP não-bloqueante na porta `9000` (`0.0.0.0:9000`).
2. Aloca um buffer de recepção de **4 MB** no kernel do sistema operacional (`SO_RCVBUF = 4194304`).
3. Carrega o segredo mestre de criptografia `ChaCha20-Poly1305`.
4. Fica em modo de escuta de alta frequência aguardando os primeiros datagramas do transmissor.

Passo 3: Conectar o Transmissor (Sender)

No computador transmissor (máquina que está capturando a tela), dispare a transmissão apontando para o IP do cliente:

```
py -3 scripts/lan_test.py --mode sender --target-ip 192.168.10.102 --port 9000 --fps 60 --bit
```

Passo 4: Leitura e Interpretação do Painel do Cliente

Assim que o transmissor inicia, o console do cliente exibirá atualizações a cada 1 segundo:

```
[+] Conexao estabelecida com Sender em: 192.168.10.50:54504
[RECV] FPS: 60.0 | Bitrate:  8.54 Mbps | Latencia LAN: 2.85 ms | Quadros: 60  | Pacotes: 780
[RECV] FPS: 60.1 | Bitrate:  8.56 Mbps | Latencia LAN: 2.62 ms | Quadros: 120 | Pacotes: 1560
[RECV] FPS: 59.9 | Bitrate:  8.51 Mbps | Latencia LAN: 2.91 ms | Quadros: 180 | Pacotes: 2340
```

Entendendo os Indicadores:

- `FPS`: Quadros por segundo efetivamente reconstruídos e prontos para decodificação (meta: 60.0).
- `Bitrate`: Taxa de transferência real em megabits por segundo (ex: 8.5 Mbps para 1080p).
- `Latência LAN`: Tempo de trânsito do quadro desde a captura até a entrega no cliente (média entre 2ms e 5ms em rede local).
- `Quadros / Pacotes`: Contadores contínuos de integridade.

Passo 5: Encerramento e Relatório Consolidado

Pressione `Ctrl+C` no cliente para encerrar a sessão. O cliente gerará o resumo consolidado:

```
================ RESUMO DO TESTE LAN ================
  Total de Quadros Montados : 1200
  Total de Pacotes Recebidos: 15600
  Volume de Dados           : 20.25 MB
  FPS Medio Efetivo         : 60.0 FPS
  Taxa de Transferencia     : 8.53 Mbps
  Latencia Media            : 2.74 ms
=====================================================
```

---

3. Fluxo Interno no Código C++ (`src/session/` e `src/packetizer/`)

Se você estiver integrando o DA-V como biblioteca C++ em sua própria aplicação cliente:

```
#include "src/core/config.hpp"
#include "src/session/session.hpp"
#include "src/packetizer/frame_assembler.hpp"

// 1. Configurar o Cliente Receptor
dav::AppConfig config = dav::AppConfig::from_default().value();
config.is_sender = false;
config.transport.bind_port = 9000;

// 2. Instanciar o FrameAssembler e a Sessão
dav::FrameAssembler assembler(200000); // 200ms deadline timeout
dav::Session session("session-lan-01", "remote-host-pc", config);

// 3. Callback de Pacote Recebido (pós-decriptação e pós-QoS)
session.set_on_packet_received([&](const dav::DavPacket& packet) {
    uint64_t now_us = dav::time::now_microseconds();

    // Tenta montar o quadro H.264
    if (auto complete_frame = assembler.push_packet(packet, now_us)) {
        // Quadro H.264 completo pronto!
        // complete_frame->data contém o bitstream Annex-B (00 00 00 01 NALs)
        // Encaminhar para o Decodificador de Vídeo ou API de Renderização
    }
});

// 4. Iniciar escuta
session.connect("0.0.0.0", 9000);
```

---

4. Troubleshooting no Cliente

Sintoma	Causa Mais Comum	Solução
Cliente não recebe pacotes	Firewall do Windows bloqueando a porta UDP	Libere a porta UDP 9000 no Firewall do Windows ou execute em prompt com permissão de rede local.
Latência acima de 20ms na LAN	Roteador com Wi-Fi 2.4 GHz congestionado	Conecte via cabo Ethernet ou rede Wi-Fi 5 GHz / 6 GHz para obter latência de ~2ms a 4ms.
Quadros incompletos / Drop	Perda severa de pacotes sem FEC suficiente	Aumente a proporção de shards FEC no `dav.toml` (`fec_parity_shards = 4`).
Falha de autenticação MAC	Chaves secretas diferentes entre Sender e Receiver	Verifique se `psk_secret` no `dav.toml` é idêntico em ambas as pontas.